

BHUVANGUARD AI

REAL-TIME AI INTERVIEW CHEAT DETECTION SYSTEM

Comprehensive System Manual, Detection Mechanics & Operating Guide

Production Status: ACTIVE & LIVE IN PRODUCTION	Version: v1.0.0 Enterprise
Live Web Application: https://bhuvan-guard-ai.vercel.app	Backend API: https://bhuvan-guard-ai.vercel.app/api
GitHub Repository: bhuvan1111/real-time-ai-interview-cheat-detection-system	Design Principle: Explainable Human-in-the-Loop

1. Executive Summary & Privacy Principles

BhuvanGuard AI is a full-stack, enterprise-grade cheat-detection platform engineered for online coding assessments and technical evaluations. While traditional surveillance software uses invasive video monitoring, eye-tracking, and keystroke logging—which violates candidate trust and induces severe testing anxiety—BhuvanGuard AI is architected from the ground up on **three non-negotiable pillars**:

1. Absolute Privacy & Trust	2. Fully Explainable Rationale	3. Human-in-the-Loop Decision
Zero webcam/audio recording, zero desktop spying. Keystroke texts and clipboard contents are never stored; only character counts and temporal deltas are captured.	Every risk score is backed by clear, plain-English evidence bullet points detailing exactly which event heuristics or anomaly thresholds were triggered.	The system never automatically accuses or disqualifies a candidate. It outputs an advisory 0–100 risk tier to empower human hiring managers to review verified evidence.

2. Complete System Architecture & Technology Stack

The system employs a reactive client-server architecture with dual channels: secure HTTPS REST for transactional calls and persistent WebSockets (with automatic REST fallback) for instantaneous event ingestion and live monitoring broadcast.

Component Layer	Technology	Key Responsibilities & Capabilities
Frontend Web App	React 18, TypeScript, Vite, Monaco Editor, Tailwind CSS	Renders candidate consent modal, code editor, safe sandbox code runner, timer, and evaluator live wall with SVG gauges and Recharts analytics.
Telemetry Engine	Custom React Hook (useMonitoring.ts)	Listens to visibilitychange, window blur, typing cadence, and paste events. Discards text content and transmits strictly numerical event metadata.
API & WebSocket Server	FastAPI, Uvicorn, ASGI WebSockets, Pydantic v2	Authenticates candidates/evaluators via JWT, handles session state, streams telemetry events, and broadcasts live risk updates to evaluator walls.
Scoring Engine	Python Heuristic Engine + scikit-learn IsolationForest	Evaluates 7 configurable weighted heuristic rules and fits an unsupervised Isolation Forest anomaly model across typing and cadence feature vectors.
Similarity Engine	Python AST Normalizer + Scikit-Learn TF-IDF	Performs dual-method code comparison: AST sequence parsing (invariant to variable renaming) and token-level TF-IDF cosine distance across historical submissions.
Persistence Layer	SQLAlchemy ORM + SQLite / PostgreSQL	Stores users, assessments, questions, telemetry events, suspicion audit logs, and code submissions with automatic schema migration.
Cloud Deployment	Vercel Serverless (Python 3.12 + Static SPA)	Global edge distribution, automatic cold-start database seeding, HTTPS encryption, and automatic fallback from WebSockets to REST polling.

3. Detection Methodology & Signal Mechanics

BhuvanGuard AI continuously analyzes **four distinct behavioral dimensions** without capturing intrusive private data:

Signal Name	Trigger Threshold	Technical Detection Mechanism	Risk Weight
Tab Switch (Brief)	≥ 1 occurrence	Browser document.visibilitychange event triggers when user switches tabs or minimizes browser window.	+5 pts
Long Tab Switch	≥ 5.0 seconds	Calculates timestamp delta (return_time - exit_time). Indicates external research or consultation of outside materials.	+10 pts
Large Text Paste	≥ 300 characters	Intercepts Monaco onPaste event. Reads event.clipboardData.getData('text').length only; the raw text is discarded.	+10 pts
Repeated Large Pastes	≥ 2 instances	Monitors cumulative paste frequency. Repeated pasting indicates multi-part external solution insertion.	+15 pts
Typing Cadence Anomaly	Score ≥ 0.70	Isolation Forest detects unnatural typing rhythms: instantaneous burst insertions without backspaces, or robotic keystroke intervals.	+10 pts
Code Similarity Match	≥ 75% structural overlap	Dual AST normalization (renamed variable matching) combined with TF-IDF cosine distance against peer candidate submissions.	+25 pts
Multi-Signal Compounding	≥ 3 distinct signals	Compound penalty applied when a candidate simultaneously shows tab switching, high paste count, and code similarity.	+10 pts

4. Risk Scoring Tiers & Explainability Matrix

Total risk scores are capped between **0 and 100** and partitioned into 4 distinct risk tiers. Every tier is associated with plain-English evaluator guidance:

Risk Tier	Score Range	Badge Color	System Interpretation & Advisory Action
LOW RISK	0 – 29	Green	Normal coding progression. Minor natural blurs or small edits. No human intervention needed.
MEDIUM RISK	30 – 59	Amber	Elevated tab switching or moderate paste activity. Evaluator should review the session timeline.
HIGH RISK	60 – 79	Orange	Multiple compound signals: rapid pasting + multiple tab switches away. Human code walkthrough recommended.
CRITICAL RISK	80 – 100	Red	Substantial anomalies detected: high code similarity (>85%) coupled with multiple multi-line pastes and extended tab switches. In-depth technical interview recommended.

5. Step-by-Step User Operating Guide

A. Candidate Flow (Taking an Assessment)

- Open the live URL: <https://bhuvan-guard-ai.vercel.app>
- Log in using candidate credentials (e.g., `alice@candidate.com` / `CandidatePass123!`).
- Review the **Informed Consent Modal** explaining exactly what data is captured (tab switch counters, paste length) and click **'I Understand & Begin'**.
- Solve algorithmic questions in the Monaco Editor with full auto-saving, syntax highlighting, and time tracking.
- Click **'Run Code'** to test against sandbox unit test cases, then click **'Submit Solution'** to finalize.

B. Evaluator Flow (Monitoring & Reviewing Candidates)

1. Log in using admin credentials: admin@interview.ai / AdminPass123!.

2. Navigate to the **Live Monitoring Wall** to view active candidate cards in real-time, complete with animated risk badges.

3. Click **'Inspect Session'** on any candidate (e.g., Diana Prince or Charlie Brown) to view:
 - **SVG Risk Gauge:** Calibrated 0–100 dial indicating exact risk tier.
 - **Explainable Evidence Card:** Itemized bullet points explaining why the score was given.
 - **Chronological Event Timeline:** Color-coded history of tab switches, focus blurs, and paste events.
 - **Code Similarity Comparison:** Side-by-side diff highlighting matching lines against peer submissions.

4. Navigate to **'Risk Rules'** to adjust threshold sensitivities or toggle specific heuristics on or off.

6. Pre-Seeded Demonstration Accounts

Role	Email Address	Password	Pre-Seeded Risk Profile & Purpose
Evaluator (Admin)	admin@interview.ai	AdminPass123!	Full access to Live Wall, Suspicion Scores, Rule Thresholds, and Analytics.
Candidate	alice@candidate.com	CandidatePass123!	LOW RISK (Score: 12) — Demonstrates standard natural coding behavior.
Candidate	bob@candidate.com	CandidatePass123!	MEDIUM RISK (Score: 42) — Moderate tab switching and occasional focus blur.
Candidate	charlie@candidate.com	CandidatePass123!	HIGH RISK (Score: 78) — Multiple large pastes and frequent extended tab switches.
Candidate	diana@candidate.com	CandidatePass123!	CRITICAL RISK (Score: 92) — Massive external paste + 91% code similarity.

7. REST API & WebSocket Endpoint Reference

Method & Endpoint	Auth Required	Description & Expected Payload
POST /api/auth/login	Public	Authenticates candidate or admin; returns JWT Bearer token and user profile.
GET /api/health	Public	Health check returning system status, service title, and DB connection state.
GET /api/assessments	Candidate / Admin	Lists all available coding assessments with question sets and time limits.
POST /api/events	Candidate	HTTP fallback to submit telemetry events (TAB_SWITCH, PASTE, TYPING_METRIC).
POST /api/submissions/run	Candidate	Executes candidate code in a secure local sandbox against test inputs.
GET /api/sessions/{id}/risk	Admin	Calculates dynamic risk score (0-100), risk tier, and itemized evidence breakdown.
WS /ws/session/{id}	Session JWT	Full duplex WebSocket stream for real-time telemetry events from candidate browser.
WS /ws/admin	Admin JWT	Broadcast WebSocket channel pushing live candidate score updates to evaluator wall.

8. Local Development & Docker Instructions

To run the system locally on your development machine:

```
# 1. Clone the repository
git clone https://github.com/bhuvan1111/real-time-ai-interview-cheat-detection-system.git
cd real-time-ai-interview-cheat-detection-system

# 2. Start Backend (Terminal 1)
cd backend && python -m venv venv && venv\Scripts\activate
pip install -r requirements.txt
python seed_data.py
uvicorn app.main:app --reload --port 8000

# 3. Start Frontend (Terminal 2)
cd frontend && npm install && npm run dev

# 4. Or Run Entire Full-Stack with Docker Compose
docker-compose up --build
```